

Sylvan: Autonomous Driving Evaluation Technology Combining Virtual and Real Environments

Project Management and Economic Analysis Report

Team Members:

- Yuxuan Ou, 2252584
- Jiye Liu, 2252752

Prepared Date: June 2026

Table of Contents

1. Project Scope
 2. Project Plan
 3. Software Process Monitoring and Control
 4. Software Process Improvement
 5. Project Risk Management
 6. Software Development and Operational Cost Estimation
 7. Pricing and Pricing Strategy
 8. Fundraising and Financing Analysis
 9. Financial Evaluation and Results
 10. Financial Risk Management
 11. Conclusion
-

1. Project Scope

1.1 Project Background

Sylvan is a non-intrusive mixed-reality evaluation platform for high-level autonomous driving systems. In end-to-end autonomous driving, the internal decision process is increasingly difficult to test through traditional modular verification. Physical proving grounds are expensive, risky, and unable to cover most long-tail hazardous scenarios. Sylvan addresses this gap by creating a CARLA-based virtual environment and synchronizing it with real vehicle motion data so that a production vehicle can be exposed to controllable virtual road hazards without modifying its original CAN bus, hardware controller, or internal autonomous driving software.

The project is positioned as a research prototype and course engineering project. Its immediate goal is not to build a full commercial SaaS product, but to demonstrate a technically feasible and economically meaningful platform for virtual-real integration, forward-facing camera testing, and AEB/FCW scenario validation. The current demo has already been operated on a real Tesla Model 3 under controlled test conditions, which strengthens the credibility of the prototype while also introducing additional safety, calibration, and site coordination requirements.

1.2 Functional Scope

1.2.1 Virtual-Real Mixed-Reality Testing

- Provide a CARLA-based virtual road environment for autonomous driving evaluation.
- Synchronize the virtual ego vehicle with physical or replayed vehicle motion data.
- Support non-intrusive testing, meaning the project does not modify the vehicle's CAN bus, original control software, or internal hardware architecture.
- Present virtual traffic scenes to the vehicle's forward-facing perception system through external visual signal injection.

1.2.2 Scenario Construction and Environmental Control

- Load CARLA built-in Town maps and OpenDRIVE `.xodr` maps.
- Build traffic scenarios containing ego vehicle, AI vehicles, pedestrians, static vehicles, and traffic cones.
- Support weather changes to simulate different visibility and lighting conditions.
- Support environment-layer control, including hiding or restoring buildings, vegetation, fences, poles, and walls.
- Provide clean-environment modes for camera calibration and visual injection experiments.

1.2.3 Vehicle Motion Data Integration

- Receive real-time vehicle pose and velocity data through ROS2.
- Support JSON replay data for offline debugging and repeatable experiments.

- Calibrate yaw offset between external vehicle data and CARLA coordinates.
- Apply external rotation and velocity information to the virtual ego vehicle.
- Maintain CARLA synchronous mode to keep world ticks and sensor frames aligned.

1.2.4 Visual Rendering and Display Injection Support

- Render forward-view virtual scenes for vehicle camera perception experiments.
- Support stereo camera mode for binocular visual layout.
- Support mono camera mode for lower-load single-view experiments.
- Display real-time simulation output through Pygame.
- Show runtime status such as speed, yaw, weather, camera mode, ROS freshness, and accident state.

1.2.5 ADAS Hazard Scenario Evaluation

- Create controllable long-tail traffic scenarios that are difficult or unsafe to reproduce physically.
- Support accident-related scenario logic such as pedestrian crossing, sudden braking, and lane-change style hazards through the accident management interface.
- Record incident statistics for later analysis.
- Focus validation on Forward Collision Warning (FCW) and Automatic Emergency Braking (AEB) reactions of a Tesla Model 3 test vehicle.

1.2.6 Runtime Operation and Experiment Control

- Provide command-line configuration for map, camera mode, data source, environment rendering, and accident simulation.
- Provide keyboard controls for weather switching, environment layer toggling, environment cleanup/restoration, yaw recalibration, and emergency exit.
- Provide optional manual vehicle control for debugging.
- Clean up CARLA actors and simulation resources after the experiment.

1.3 Non-functional Scope

1.3.1 Real-time Performance

The project targets a closed-loop latency of no more than 50 ms. The current software design supports this objective through CARLA synchronous mode, fixed simulation FPS, direct sensor queue synchronization, and lightweight per-frame processing.

1.3.2 Reliability and Cleanup

CARLA actors, sensors, hidden environment objects, ROS nodes, and Pygame resources are cleaned up explicitly.

1.3.3 Safety

Sylvan is designed for non-intrusive signal injection and external synchronization, but the system still interacts with real AEB/FCW behavior, so physical safety control is mandatory.

2. Project Plan

2.1 Project Overview Statement

POS Item	Content
Project Name	Sylvan: Autonomous Driving Evaluation Technology Combining Virtual and Real Environments
Project Start and Finish Date	March 24, 2026 to June 16, 2026
Project Manager	Jiye Liu, 2252752
Core Team	Jiye Liu, 2252752; Yuxuan Ou, 2252584
Problem or Opportunity	End-to-end autonomous driving systems are increasingly difficult to verify through traditional modular testing. Physical proving grounds are costly and cover only a small part of possible hazardous scenarios. A high-fidelity and non-intrusive virtual-real testing approach is needed.
Goal	Build a non-intrusive mixed-reality testing platform that synchronizes a CARLA virtual vehicle with real or replayed vehicle motion data and presents high-fidelity virtual scenes for autonomous driving evaluation.
Objectives	Within 1 month, complete single-perspective front-view mixed-reality camera integration. Within 2.5 months, complete the Sylvan Demo System and trigger AEB/FCW reactions on a Tesla Model 3. In the final 0.5 month, conduct system testing and validation.
Success Criteria	Virtual signal injection success rate at least 70%; closed-loop latency no more than 50 ms; FCW scene recognition accuracy at least 70%.
Assumptions	Existing laboratory hardware, Tesla vehicle platform, IMU device, CARLA simulator, and ROS environment are available. The external display/injection method can provide usable visual signals without modifying the vehicle's internal systems.

Major Risks	Hardware compatibility, latency, signal quality, safety during vehicle tests, schedule pressure, and evolving autonomous driving test requirements.
Approved Budget	RMB 0 direct external budget; existing laboratory hardware, vehicle platform, and open-source software resources are reused.
Prepared By	Yuxuan Ou, 2252584, March 19, 2026
Approved By	Jiye Liu, 2252752, March 19, 2026

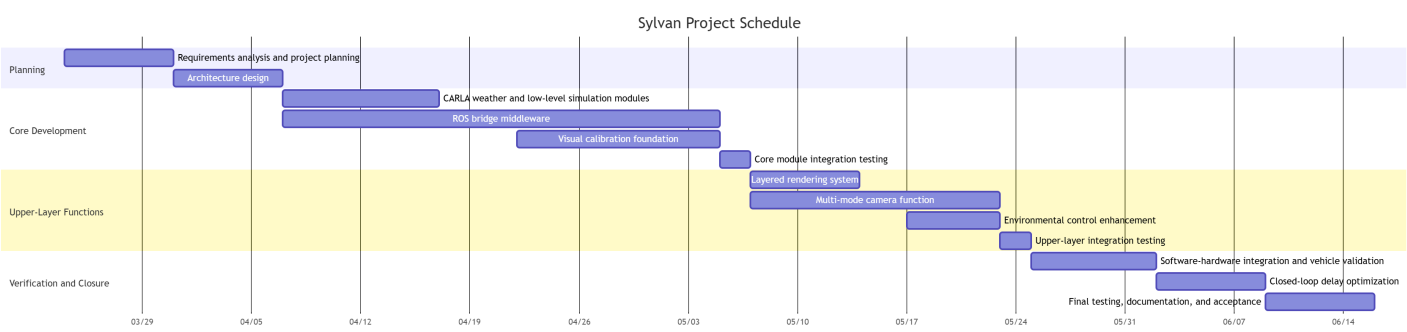
2.2 Work Breakdown Structure

WBS ID	Work Package	Main Deliverables
1.0	Project Planning and Requirements	Project overview statement, project charter, scope boundary, success criteria
2.0	System Architecture	Virtual-real integration architecture, module boundaries, interface design
3.0	Core Simulation Platform	CARLA connection, synchronous mode, map loading, OpenDRIVE support
4.0	Vehicle Motion Synchronization	ROS bridge, JSON replay, yaw calibration, velocity and pose synchronization
5.0	Visual Rendering Pipeline	Forward-view rendering, mono/stereo camera modes, HUD, display output
6.0	Scenario and Environment Control	Weather control, layered rendering, traffic vehicles, pedestrians, static objects, accident scenarios
7.0	Software-Hardware Integration	External visual injection, IMU data flow, Tesla Model 3 closed-loop verification
8.0	Documentation and Economic Analysis	Size measurement, cost estimation, management report, final acceptance materials

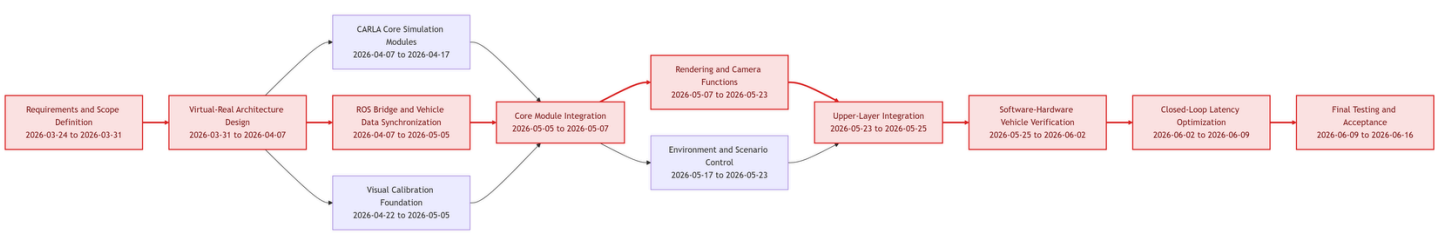
2.3 Milestone Schedule

Phase	Date Range	Major Work
Phase 1	2026-03-24 to 2026-03-31	Requirements analysis, scope definition, project planning

Phase 2	2026-03-31 to 2026-04-07	Architecture design for virtual-real integration
Phase 3	2026-04-07 to 2026-05-07	CARLA custom weather, ROS bridge, data synchronization, visual calibration foundation
Phase 4	2026-05-07 to 2026-05-25	Layered rendering, mono/stereo camera modes, environmental control enhancement
Phase 5	2026-05-25 to 2026-06-02	Software-hardware integration and vehicle-level verification
Phase 6	2026-06-02 to 2026-06-09	Closed-loop latency optimization and repeated validation
Phase 7	2026-06-09 to 2026-06-16	Final bug fixing, documentation, acceptance, and closure



2.4 Critical Path



Critical activities are architecture design, CARLA core integration, ROS vehicle data synchronization, visual calibration, camera rendering, software-hardware integration, and latency optimization. Any delay in these activities directly delays vehicle-level verification and final acceptance.

2.5 Deliverables

- A modular Python software package for CARLA Auto VR simulation.
- CLI entry scripts for synchronized simulation and ROS bridge operation.
- CARLA world, vehicle, traffic, camera, weather, and environment-layer control modules.

- ROS2 and JSON data source adapters.
 - Accident scenario management and statistics interface.
 - Function point measurement report.
 - Software cost estimation report.
 - SRS and SDS
 - Project management and economic analysis report.
-

3. Software Process Monitoring and Control

3.1 Project Tracking Tool

- **Git version control:** Used to track code and document changes, preserve rollback capability, and separate cleanup/documentation work from functional implementation work.
- **Milestone checklist:** Used to compare actual progress with the seven planned phases from the project charter.
- **Documentation tracker Feishu:** Used to maintain the Project Overview Statement, Project Charter, software size measurement report, cost estimation report, and final management report.
- **Integration checklist:** Used before demonstrations to confirm that the CARLA environment, ROS/JSON data source, display mode, scenario setting, and safety conditions are ready.
- **Meeting and communication record:** Used for short progress synchronization, task assignment, issue confirmation, and final acceptance preparation.

3.2 Development Methodology

The project uses a lightweight agile development method.

3.2.1 Iterative Milestone Development

- Phase 1: Requirements, project scope, and success criteria were defined.
- Phase 2: The virtual-real integration architecture and module boundaries were designed.
- Phase 3: Core modules such as CARLA integration, ROS bridge, weather control, and calibration foundation were developed.
- Phase 4: Upper-layer functions such as layered rendering, camera modes, and environmental controls were integrated.
- Phase 5: Software-hardware integration and vehicle-level verification were carried out.
- Phase 6: Closed-loop latency optimization was performed.

- Phase 7: Final testing, report writing, and acceptance preparation were completed.

3.2.2 Version Control Practice

- Code and documentation were maintained through Git.
- Project cleanup work, report writing, and code modification were kept traceable through the working tree.
- Stable deliverables were preserved in Markdown documents under `docs/`.

3.3 Earned Value Control

Work Package	Planned Value Weight	Completion Evidence
Planning and requirements	10%	Project overview statement and charter completed
Architecture and modularization	15%	<code>carla_auto_vr</code> package structure implemented
CARLA connection and synchronization	15%	Client, sync mode, map loader implemented
Data source integration	15%	ROS bridge, JSON player, yaw calibration implemented
Camera and UI rendering	15%	Stereo/mono rigs, Pygame display, HUD implemented
Environment and traffic control	15%	Weather, layered renderer, traffic modules implemented
Accident and vehicle verification	10%	Accident manager and statistics interface implemented
Documentation and closure	5%	Size, cost, and management reports prepared

4. Software Process Improvement

4.1 Process Improvement Objectives

To improve delivery efficiency, experiment reliability, and team collaboration quality, the Sylvan project adopts a continuous software process improvement strategy. The improvement work

focuses on reducing rework caused by unclear requirements, improving the predictability of simulator-based integration, and forming reusable project assets for future vehicle-safety experiments.

4.1.1 Main Objectives

- Improve on-time delivery rate: Keep the planned milestone completion rate above 90%.
- Reduce rework rate: Reduce repeated modification caused by unclear scenarios, inconsistent environments, or late requirement changes by 40%.
- Improve quality control efficiency: Move review and validation activities earlier in the process so that integration defects can be found before final simulator testing.
- Enhance team collaboration effectiveness: Clarify task ownership, review responsibility, and communication rhythm among requirement, development, testing, and documentation work.
- Accumulate reusable process assets: Form standardized templates for scenario definition, experiment checklist, defect recording, and project closure.

4.1.2 Specific Indicators

- Schedule variance is controlled within +/-10%.
- Requirement changes are recorded and assessed within 48 hours.
- Major integration defects found during final verification are reduced by 30%.
- All core deliverables have traceable evidence, including requirement source, implementation owner, validation record, and report reference.
- A standardized and measurable project process is established to support later scenario expansion and team knowledge transfer.

4.2 Process Improvement Method

4.2.1 Improvement of Requirement Management Process

- Improvement Strategy:
 - Establish a lightweight change control mechanism for scenario, interface, and experiment requirement changes.
 - Manage requirement traceability from POS, project charter, WBS, schedule, implementation evidence, and final report.
 - Introduce acceptance criteria for each major work package before implementation starts.
- Specific Measures:
 - Use a standardized requirement and scenario description template.

- Record the impact of each requirement change on schedule, risk, and verification workload.
- Hold regular requirement review meetings before high-risk integration stages.

4.2.2 Standardize the Design Process

- Improvement Strategy:
 - Establish design review checkpoints before implementation and before integration testing.
 - Standardize the documentation of module responsibility, external dependency, configuration item, and verification method.
 - Strengthen version control of design documents and management reports.
- Specific Measures:
 - Review architecture decisions together with the corresponding WBS task and milestone.
 - Maintain a clear mapping between design scope, work package, responsible member, and validation evidence.
 - Treat CARLA, ROS, data replay, rendering, and accident verification as separately reviewable integration units instead of one large final-stage task.

4.2.3 Team Management Optimization

- Improvement Strategy:
 - Establish regular progress synchronization and workload balancing.
 - Improve cross-functional collaboration among development, experiment validation, and documentation roles.
 - Convert project experience into reusable knowledge assets.
- Specific Measures:
 - Use weekly milestone review to compare planned value, earned value, and actual effort.
 - Assign clear owners for requirements, simulator integration, data source validation, report writing, and final acceptance.
 - Maintain a project knowledge base covering environment setup, common failure causes, validation checklist, and lessons learned.

4.2.4 Test Process Improvement

- Improvement Strategy:
 - Shift testing activities left by defining validation criteria together with requirements.
 - Combine manual simulator verification with lightweight repeatable checks.

- Improve the defect management process from discovery to confirmation and closure.
- Specific Measures:
 - Prepare smoke-test checklists for environment startup, data replay, camera display, traffic generation, and accident statistics.
 - Classify defects by requirement defect, environment defect, integration defect, and documentation defect.
 - Review test evidence before milestone acceptance instead of waiting until project closure.

4.3 Process Improvement Examples and Results

Improvement Example: Simulator Integration Efficiency Improvement

1. Problems Before Improvement:

- Integration tasks were concentrated in the later stage, creating schedule pressure.
- Environment differences could cause repeated verification work.
- Scenario requirements and test evidence were not always recorded in the same format.

2. Improvement Measures:

- Split simulator integration into independently reviewable work packages.
- Introduced milestone-based validation checklists for data source, rendering, environment control, traffic generation, and accident verification.
- Used progress tracking and earned value analysis to identify mid-stage effort deviation.
- Standardized the relationship among requirement source, implementation evidence, and report content.

3. Improvement Effect:

- Integration risk became visible earlier instead of being discovered only during final testing.
- Rework caused by unclear validation criteria was reduced.
- Project documentation became easier to audit because each major deliverable had corresponding management evidence.
- The team formed reusable process assets for later FCW/AEB scenario expansion and simulator-based verification.

4.4 Follow-up Improvement Direction

1. Improve the quality measurement system:

- Add quantitative defect statistics by stage and defect type.

- Establish review pass-rate and rework-rate indicators.
 - Track requirement stability and change response time.
2. Optimize project monitoring mechanism:
- Continue using Gantt chart, milestone chart, critical path tracking, and earned value analysis.
 - Establish an early-warning rule when schedule variance or effort variance exceeds the control threshold.
 - Keep a weekly progress record for planned work, completed work, open risks, and next actions.
3. Establish organizational process assets:
- Form standardized templates for scenario requirements, smoke testing, defect tracking, and project closure.
 - Build a best-practice knowledge base for CARLA/ROS experiment setup and validation.
 - Conduct process compliance review before final acceptance and project closing.
-

5. Project Risk Management

5.1 Risk Identification

Risk Type	Risk Item	Possible Cause	Impact	Probability
Technical	CARLA version incompatibility	CARLA 0.9.15 Python egg path or Python version mismatch	High	Medium
Technical	ROS2 custom message unavailable	Missing testcarla_inte rfaces workspace or wrong environment path	High	Medium
Technical	Closed-loop latency above 50 ms	Rendering load, ROS delay, sensor queue timeout, GPU pressure	High	Medium

Technical	Configuration dependency missing	Scenario constants or map safety settings not available in runtime environment	High	Medium
Technical	Pedestrian navmesh crash	Certain maps lack pedestrian navigation support	High	Medium
Hardware	Display injection quality insufficient	Screen brightness, camera exposure, optical alignment, refresh rate	High	Medium
Safety	Real vehicle test hazard	AEB/FCW scenarios affect real vehicle behavior	High	Low to Medium
Schedule	Integration takes longer than planned	CARLA, ROS, Pygame, and hardware debugging are coupled	Medium	Medium
Cost	Workstation or device cost exceeds estimate	GPU requirement and external display equipment	Medium	Medium
Management	Documentation lags behind code	Rapid iteration near final deadline	Medium	Medium

5.2 Risk Monitoring Mechanism

- Verify CARLA import and server connection before each run.
- Verify ROS workspace and custom message import before vehicle experiments.
- Use the main loop FPS log as a basic performance indicator.
- Use ROS data freshness checks to identify stale data.
- Use map blacklist and navmesh probing before enabling accident simulation.
- Review Git status and documentation before final submission.

5.3 Risk Response Plan

Risk	Response
CARLA import failure	Provide explicit CARLA egg search paths and installation instructions
ROS bridge failure	Fall back to JSON replay for offline verification
Latency exceeds target	Reduce rendering complexity, use mono camera mode, hide unnecessary environment objects, optimize frame processing
Pedestrian navmesh unavailable	Disable accident mode automatically unless explicitly forced
Vehicle test unsafe	Use closed proving ground, low-speed scenarios, safety driver, and staged testing
Hardware injection unstable	Calibrate display position, brightness, FOV, and camera alignment before formal testing
Schedule pressure	Prioritize core closed-loop demonstration over nonessential features
Cost overrun	Reuse laboratory vehicle, IMU, workstation, and open-source CARLA resources

6. Software Development and Operational Cost Estimation

6.1 Software Prototype Size Estimation: Function Point Analysis

6.1.1 Data Function Points (ILF/EIF)

Type	Logical File	Data Elements (DET)	Record Elements (RET)	Complexity	Weight
ILF	Simulation Settings and Scenario Configuration	host, port, map, camera mode, render switches, accident thresholds, traffic limits, spawn coordinates, OpenDRIVE	5	Medium	10

		parameters, logging level			
ILF	Runtime Actor Registry and World State	actor id, tag, CARLA type, lifecycle state, sensor reference, hidden object id, traffic role	3	Low	7
ILF	Vehicle Synchronization State	timestamp, roll, pitch, yaw, velocity x/y/z, target transform, yaw offset, freshness flag	2	Low	7
ILF	Accident Scenario State and Statistics	active scenario, cooldown, nearby actors, timers, participant references, incident result counters	3	Medium	10
ILF	Sensor, Display, and HUD State	camera mode, FOV, resolution, surfaces, FPS, speed, weather label, keyboard state	2	Low	7
EIF	CARLA Simulator World and Blueprint Library	map data, spawn points, weather presets, actor blueprints, traffic manager state, city object labels	4	Medium	7
EIF	ROS2 Vehicle Data Topic	timestamp, pitch, roll, yaw, velocity x/y/z, topic name, QoS metadata, freshness time	2	Low	5
EIF	External JSON Replay and OpenDRIVE Files	replay frames, rotation, velocity, map geometry path, file validity, generation parameters	2	Low	5

6.1.2 Transaction Function Points (EI/EO/EQ)

Type	Transaction	Data Elements (DET)	FTR	Complexity	Weight
EI	CLI Simulation Start and Configuration	host, port, map, camera mode, ROS/JSON switch, debug flag, environment flags, accident flag	3	Medium	4
EI	ROS Vehicle Data Intake	timestamp, pitch, roll, yaw, velocity x/y/z, topic source	2	Medium	4
EI	JSON Frame Replay Intake	timestamp, rotation fields, velocity fields, file path, frame index	2	Medium	4
EI	Keyboard Control and Runtime Commands	WASD, reverse, weather, layer toggles, camera info, yaw recalibration, quit command	4	High	6
EI	Map Selection and OpenDRIVE Import Request	built-in map name, <code>.xodr</code> path, search paths, generation parameters	2	Medium	4
EI	Accident Mode Selection and Trigger Control	auto/forced/disabled flag, map safety check, speed threshold, trigger interval	3	Medium	4
EO	Camera Frame Rendering	CARLA image frame, stereo/mono layout, surfaces, FOV, display resolution	3	High	7
EO	HUD and Status Rendering	speed, FPS, yaw, weather, camera mode, ROS freshness, accident state	3	Medium	5
EO	CARLA World and Traffic Actor	blueprints, spawn transforms, autopilot	4	High	7

	Generation	settings, walker controllers, traffic manager rules			
EO	Environment Layer Hide/Restore	object type, object ids, render status, CARLA city labels	2	Medium	5
EO	Accident Scenario Execution Output	scenario type, target actors, motion control, recovery, statistics update	4	High	7
EO	Logging and Diagnostic Output	startup logs, topic status, error stack, FPS logs, cleanup count	2	Low	4
EQ	Layer Status and Camera Information Query	layer booleans, camera mode, FOV, current render state	2	Medium	4
EQ	ROS Topic and Data Freshness Query	available topics, last update time, signal count, freshness threshold	2	Medium	4
EQ	Map Navigation Capability Check	map name, OpenDRIVE flag, blacklist result, navigation probe result	2	Low	3
EQ	Resource Cleanup Summary Query	actor tags, actor count, destroy result, cleanup status	1	Low	3

6.1.3 Measure Function Point Complexity

According to NESMA standards, function point complexity measurement must identify data functions and transaction functions separately. For Sylvan, the complexity is mainly caused by heterogeneous real-time interfaces, CARLA actor lifecycle control, sensor rendering, and external vehicle data synchronization.

6.1.3.1 Internal Logical File (ILF) Complexity

--	--	--	--	--

Logical File	DET	RET	Complexity	Weight
Simulation Settings and Scenario Configuration	20+	5	Medium	10
Runtime Actor Registry and World State	10-15	3	Low	7
Vehicle Synchronization State	8-10	2	Low	7
Accident Scenario State and Statistics	20+	3	Medium	10
Sensor, Display, and HUD State	10-15	2	Low	7

6.1.3.2 External Interface File (EIF) Complexity

Logical File	DET	RET	Complexity	Weight
CARLA Simulator World and Blueprint Library	20+	4	Medium	7
ROS2 Vehicle Data Topic	8-12	2	Low	5
External JSON Replay and OpenDRIVE Files	8-12	2	Low	5

6.1.3.3 External Input (EI) Complexity

Transaction	DET	FTR	Complexity	Weight
CLI Simulation Start and Configuration	10+	3	Medium	4
ROS Vehicle Data Intake	7	2	Medium	4
JSON Frame Replay Intake	7	2	Medium	4
Keyboard Control and Runtime Commands	10+	4	High	6
Map Selection and OpenDRIVE Import	6	2	Medium	4

Request				
Accident Mode Selection and Trigger Control	6	3	Medium	4

6.1.3.4 External Output (EO) Complexity

Transaction	DET	FTR	Complexity	Weight
Camera Frame Rendering	10+	3	High	7
HUD and Status Rendering	8	3	Medium	5
CARLA World and Traffic Actor Generation	15+	4	High	7
Environment Layer Hide/Restore	6	2	Medium	5
Accident Scenario Execution Output	10+	4	High	7
Logging and Diagnostic Output	6	2	Low	4

6.1.3.5 External Query (EQ) Complexity

Transaction	DET	FTR	Complexity	Weight
Layer Status and Camera Information Query	4	2	Medium	4
ROS Topic and Data Freshness Query	5	2	Medium	4
Map Navigation Capability Check	4	2	Low	3
Resource Cleanup Summary Query	3	1	Low	3

6.1.4 NESMA Function Point Calculation

6.1.4.1 Unadjusted Function Points (UFP)

Data Function Points (58) + Transaction Function Points (75) = 133 FP.

Data Function Points:

Type	Component	Count	Weight	Subtotal
ILF	Simulation Settings and Scenario Configuration	1	10	10
ILF	Runtime Actor Registry and World State	1	7	7
ILF	Vehicle Synchronization State	1	7	7
ILF	Accident Scenario State and Statistics	1	10	10
ILF	Sensor, Display, and HUD State	1	7	7
EIF	CARLA Simulator World and Blueprint Library	1	7	7
EIF	ROS2 Vehicle Data Topic	1	5	5
EIF	External JSON Replay and OpenDRIVE Files	1	5	5
Total				58

Transaction Function Points:

Type	Component	Count	Weight	Subtotal
EI	CLI Simulation Start and Configuration	1	4	4
EI	ROS Vehicle Data Intake	1	4	4
EI	JSON Frame Replay Intake	1	4	4
EI	Keyboard Control and Runtime Commands	1	6	6
EI		1	4	4

	Map Selection and OpenDRIVE Import Request			
EI	Accident Mode Selection and Trigger Control	1	4	4
EO	Camera Frame Rendering	1	7	7
EO	HUD and Status Rendering	1	5	5
EO	CARLA World and Traffic Actor Generation	1	7	7
EO	Environment Layer Hide/Restore	1	5	5
EO	Accident Scenario Execution Output	1	7	7
EO	Logging and Diagnostic Output	1	4	4
EQ	Layer Status and Camera Information Query	1	4	4
EQ	ROS Topic and Data Freshness Query	1	4	4
EQ	Map Navigation Capability Check	1	3	3
EQ	Resource Cleanup Summary Query	1	3	3
Total				75

Therefore:

UFP = Data Function Points + Transaction Function Points = 58 + 75 = 133 FP.

6.1.4.2 Adjustment Factor Calculation

Based on the 14 General System Characteristic (GSC) ratings from the size measurement report:

No.	GSC Factor	Rating	Rationale

1	Data Communication	5	Real-time interaction with ROS2 topics and CARLA simulator
2	Distributed Data Processing	4	CARLA server, Python client, ROS2 node, and Pygame display cooperate across processes
3	Performance Requirements	5	The simulation loop depends on stable FPS and timely sensor frame processing
4	Heavily Used Configuration	4	Multiple CLI flags, scenario constants, map choices, and runtime switches
5	Transaction Frequency	4	Vehicle state, sensor frames, HUD, and keyboard events update continuously
6	Online Data Entry	3	Runtime commands are entered through keyboard and command-line options
7	End-User Efficiency	4	Hotkeys, HUD, clean-environment mode, and map safety checks improve operation efficiency
8	Online Updates	4	Vehicle transform, velocity, weather, traffic, and environment layers update during execution
9	Complex Processing Logic	4	Includes yaw calibration, accident triggering, traffic actor

			generation, and map compatibility checks
10	Code Reusability	3	Modular domains and adapters reduce duplicated implementation
11	Ease of Installation	2	Python package setup is simple, but CARLA/ROS2 environment setup is heavy
12	Ease of Operation	3	CLI and keyboard shortcuts are available, but simulator startup is still required
13	Multi-Site Deployment	1	Mainly deployed on a local simulation workstation
14	Change Adaptability	3	Centralized configuration helps, but simulator APIs and scenario logic still require careful changes

The total degree of influence is:

$$TDI = 5 + 4 + 5 + 4 + 4 + 3 + 4 + 4 + 4 + 3 + 2 + 3 + 1 + 3 = 49.$$

According to the NESMA adjustment formula:

$$VAF = 0.65 + 0.01 \times TDI = 0.65 + 0.01 \times 49 = 1.14.$$

Adjusted Function Points:

$$AFP = UFP \times VAF = 133 \times 1.14 = 151.62, \text{ rounded to } 152 \text{ FP.}$$

6.1.5 Experiment Results and Analysis

The final function point measurement results are summarized below.

Metric	Result	Description
--------	--------	-------------

Unadjusted FP (UFP)	133 FP	Data FP (58) + Transaction FP (75)
Adjustment Factor (VAF)	1.14	Based on 14 GSC factors (TDI = 49)
Adjusted FP (AFP)	152 FP	$133 \times 1.14 = 151.62$, rounded to nearest integer
NESMA Estimated Method Validation	137 FP	Estimated AFP = $120 \times 1.14 = 136.8$, rounded to 137 FP
Deviation	9.9%	Absolute difference between 152 FP and 137 FP divided by 152 FP; acceptable because it is below 20%

The detailed method is selected as the main result because Sylvan is not a typical CRUD information system. It includes real-time CARLA simulator control, ROS2/JSON vehicle data ingestion, camera and HUD rendering, map loading, traffic actor lifecycle management, environment layer control, and accident scenario behavior. These functions have different complexity levels and are better represented by detailed counting.

The NESMA Estimated Method is used as a validation method. Its result differs from the detailed result by approximately 9.9%, which supports the reasonableness of using 152 FP as the software size input for cost estimation.

The software cost estimation report further converts this software size into effort and cost:

Item	Lower Bound	Upper Bound	Basis
Productivity Standard	3.88 HH/FP	6.83 HH/FP	P25 and P50 productivity benchmarks
Development Effort	589.76 person-hours	1,038.16 person-hours	152 FP x productivity standard
Person-Month Conversion	3.28 PM	5.77 PM	1 PM = 180 person-hours
System Adjustment Factor	1.09	1.09	Composite system characteristic adjustment

Adjusted Effort	3.57 PM	6.29 PM	Effort x 1.09
Labor Cost	RMB 111,450.32	RMB 196,187.04	Shanghai P50 rate: RMB 31,207/person-month
Direct Non-Labor Cost	RMB 7,400.00	RMB 7,400.00	Workstation, display equipment, storage, and documentation resources
Total Estimated Software Cost	RMB 118,850.32	RMB 203,587.04	Labor cost + direct non-labor cost

The estimated cost is reasonable for this project because Sylvan is an integration-heavy real-time simulation system. Its development effort is not only determined by code volume, but also by CARLA compatibility, ROS2 communication, sensor rendering, vehicle synchronization, map loading, traffic simulation, runtime cleanup, and hardware-dependent testing.

7. Pricing and Pricing Strategy

7.1 Pricing Position

The project is extended into a pilot product for laboratories, OEMs, or autonomous driving suppliers, pricing should reflect the value of reducing physical proving-ground cost and improving long-tail scenario coverage.

7.2 Target Customers

Potential customers include:

- University autonomous driving laboratories.
- Vehicle OEM testing departments.
- ADAS and autonomous driving suppliers.
- Simulation and validation service providers.
- Research groups requiring controllable AEB/FCW experiments.

7.3 Proposed Pricing Model

Product or Service	Suggested Price Range	Description
--------------------	-----------------------	-------------

Research pilot deployment	RMB 80,000 to 120,000 per project	One-time deployment for a laboratory or vehicle platform
Annual laboratory license	RMB 150,000 to 220,000 per year	Software updates, scenario library, documentation, and support
Custom scenario package	RMB 20,000 to 50,000 per package	Custom FCW/AEB, weather, map, and traffic scenarios
On-site integration support	RMB 1,500 to 2,500 per person-day	Vehicle calibration, CARLA setup, test support
Academic collaboration package	Negotiated	Lower direct price, with joint publication or research output

7.4 Pricing Strategy

The recommended pricing strategy is value-based plus pilot-friendly.

1. Use low-friction pilot pricing for first laboratory customers.
2. Offer discounted academic collaboration to build credibility.
3. Price custom scenarios separately because they require domain-specific engineering.
4. Keep deployment support billable because hardware integration varies by vehicle.
5. Avoid promising full production-grade reliability before sufficient vehicle test evidence is collected.

7.5 Break-even Implication

Using the upper-bound professional-equivalent cost of RMB 203,587.04, the project can recover its development cost through:

- Two pilot deployments at approximately RMB 100,000 each plus support, or
- One annual laboratory license plus one custom scenario package, or
- A funded research continuation project above RMB 200,000.

8. Fundraising and Financing Analysis

8.1 Current Funding Status

There is no direct external commercial funding in the current course phase. The direct approved budget is RMB 0, while actual resources are provided by the laboratory and team effort.

8.2 Financing Need for Future Development

If Sylvan continues beyond, funding will be needed for:

- More stable real-vehicle testing.
- Additional display and optical injection devices.
- Dedicated GPU workstation resources.
- Scenario library development.
- Automated latency measurement and data recording.
- Safety certification and experiment site cost.

8.3 Recommended Funding Sources

Source	Suggested Amount	Purpose
University research fund	RMB 100,000 to 200,000	Continue prototype validation and documentation
Laboratory equipment support	In-kind	Vehicle, IMU, workstation, display, test site
Industry pilot sponsorship	RMB 200,000 to 500,000	Vehicle-specific integration and scenario testing
Innovation grant	RMB 300,000 to 800,000	Build repeatable platform and scenario library
Strategic OEM collaboration	Negotiated	Joint validation and test data collection

8.4 Financing Strategy

The project should avoid heavy debt financing in the early stage because technical and market uncertainty remain high. The preferred financing path is:

1. Complete academic prototype acceptance.
2. Obtain laboratory or university seed funding.
3. Run one or two industry pilots.
4. Use pilot results to apply for innovation grants or strategic sponsorship.

5. Consider commercial company formation only after repeatable vehicle-level evidence is obtained.

8.5 Investor Value Proposition

Sylvan's value proposition is:

- Lower cost than constructing many physical hazardous scenarios.
 - Safer validation of long-tail ADAS and autonomous driving behavior.
 - Non-intrusive approach that avoids CAN bus reverse engineering.
 - High-fidelity CARLA scene generation with real vehicle motion synchronization.
 - Expandable architecture for future sensor modalities and scenario libraries.
-

9. Financial Evaluation and Results

9.1 Basic Project Information

The current course project has no approved external cash budget because the team reuses laboratory equipment, open-source software, and existing vehicle resources. For financial evaluation, however, the project should be assessed by its professional-equivalent cost and future pilot commercialization potential. Since the demo has already been operated on a real Tesla Model 3, the evaluation also includes vehicle-level validation preparation instead of treating the system as a pure simulator prototype.

9.2 Project Construction Investment Table

9.2.1 Software Development and Maintenance Costs

The upper-bound professional-equivalent cost from the software cost estimation report is used as the conservative baseline for productization evaluation.

Item	Amount (RMB)	Basis
Software development professional-equivalent cost	203,587.04	Upper-bound result from Section 6
Subtotal	203,587.04	Used as the initial software construction investment

9.2.2 Equipment and Tool Purchase Costs

Direct non-labor cost at RMB 7,400, covering workstation depreciation, display equipment, storage, and documentation resources. Because the current demo has been run on a real vehicle, the final evaluation adds vehicle-level validation preparation cost.

Item	Amount (RMB)	Description
Workstation, display, storage, and documentation resources	7,400.00	Direct non-labor cost from Experiment III Section 2.7
Vehicle calibration, test-site preparation, and safety support	35,000.00	Additional cost required by real-vehicle demo operation
Subtotal	42,400.00	Equipment, tools, and validation preparation

9.2.3 Basic Reserve Funds

The basic reserve fund is calculated as 10% of software development cost plus equipment/tool/validation preparation cost.

Reserve Fund = (203,587.04 + 42,400.00) x 10% = RMB 24,598.70

10. Financial Risk Management

10.1 Revenue Sensitivity Analysis

Scenario: impact of revenue fluctuation on NPV, IRR, and payback period while keeping the baseline operating cost unchanged.

Revenue Change	-20%	-10%	Baseline	0.1	0.2
NPV (RMB 10,000)	0.46	20.47	40.49	60.51	80.53
IRR	8.39%	23.62%	36.25%	47.34%	57.43%
Static Payback Period (Years)	4.42	3.53	2.97	2.51	2.23

Key Insights:

- At a 20% revenue decline, NPV remains slightly positive, but IRR is close to the 8% discount rate. This means revenue uncertainty is the most sensitive financial factor.

- Every 10% revenue decrease materially delays payback, so the project must secure early paid pilots or research sponsorship before expanding the operating team.
- The existing real-vehicle demo helps reduce revenue risk because it provides stronger validation evidence than a pure simulator demonstration.

10.2 Cost Sensitivity Analysis

Scenario: impact of operating cost deviation on financial indicators while keeping baseline revenue unchanged.

Cost Change	-15%	-5%	Baseline	0.05	0.15
NPV (RMB 10,000)	60.12	47.03	40.49	33.95	20.87
IRR	47.22%	40.03%	36.25%	32.31%	23.85%
Static Payback Period (Years)	2.52	2.79	2.97	3.12	3.52

Critical Risks:

- A 15% cost increase still leaves the project financially feasible, but NPV decreases by nearly half compared with the baseline.
- The most important cost drivers are vehicle-specific integration, test-site coordination, display calibration, and scenario customization.
- Direct non-labor resources from Experiment III Section 2.7 are small, but real-vehicle operation can quickly increase field support and safety preparation costs.

10.3 Compound Risk Simulation

Stress Test: 10% revenue decline, 10% operating cost increase, and a higher 10% discount rate.

Metric	Baseline	Stress Scenario
NPV (RMB 10,000)	40.49	4.72
IRR	36.25%	14.07%
Dynamic Payback Period (Years)	3.26	4.68
Financial Judgment	Attractive	Still feasible but close to the warning zone

Action Hierarchy:

- 1. Revenue Stability: Sign at least one paid pilot or funded research continuation before scaling the team.
- 2. Cost Control: Standardize vehicle calibration, display injection setup, and scenario package delivery.
- 3. Cash Buffer: Maintain reserve funding for test-site delay, hardware replacement, and safety support.

10.4 Risk Mitigation Optimization

Risk Type	Financial Impact	Countermeasures
Revenue volatility	Lower NPV and delayed payback	Use real-vehicle demo evidence to secure research pilots, annual laboratory licenses, and staged OEM collaboration
Vehicle integration cost overrun	Higher operating cost and lower margin	Define a standard vehicle setup checklist, limit custom work scope, and bill on-site support separately
Real-vehicle safety incident	Severe financial, legal, and reputational loss	Use closed-site testing, safety driver, low-speed staged validation, emergency stop plan, and formal pre-test checklist
Hardware and test-site cost increase	Higher upfront cash need	Reuse laboratory resources, procure equipment in stages, and separate optional hardware cost from software license price
Funding delay	Insufficient working capital for pilot expansion	Prioritize grants, university research funds, and industry sponsorship before considering debt financing
Latency or performance instability	Lower customer willingness to pay	Track latency as a formal acceptance metric and provide lower-load mono camera mode for constrained environments

10.5 Sensitivity Conclusions

- 1. Resilience: Under baseline assumptions, the project has positive NPV and acceptable payback. It can tolerate moderate cost growth, but it is highly sensitive to revenue decline.
- 2. Strategic Levers: The most effective financial controls are paid pilot acquisition, standardized vehicle integration, reusable scenario packages, and strict field-test cost management.

3. Thresholds: If expected revenue falls by more than 20% or vehicle integration cost rises by more than 15% for two consecutive pilot projects, the project should delay commercial expansion and return to research-funded development.
 4. Final Judgment: Because the demo has already operated on a real vehicle, Sylvan has stronger commercialization evidence than a simulator-only prototype. The remaining financial risk is not technical feasibility alone, but whether the team can turn the real-vehicle demo into a repeatable, controllable, and safely deliverable pilot service.
-

11. Conclusion

11.1 Project Closure

Sylvan has established a modular CARLA-based mixed-reality autonomous driving evaluation platform. The current codebase supports CARLA connection, synchronous simulation, map loading, vehicle spawning, ROS2/JSON vehicle data synchronization, yaw calibration, mono/stereo camera rendering, Pygame display, HUD, keyboard controls, weather switching, layered environment control, traffic generation, and accident management.

The project scope is aligned with the original charter: it focuses on single forward-view camera virtual-real integration and avoids intrusive vehicle modification. The system is suitable for demonstrating FCW/AEB-oriented scenario validation in a controlled research environment.

11.2 Economic Summary

The NESMA Detailed Method estimates the software size at 152 adjusted function points. Based on productivity standards and system adjustment factors, the professional-equivalent effort is 3.57 to 6.29 person-months. The total estimated software cost is RMB 118,850.32 to RMB 203,587.04 after adding direct non-labor costs.

For the current course project, the approved direct budget is RMB 0 because the team reuses laboratory resources. For future commercialization, the project may recover development cost through pilot deployments, annual laboratory licenses, and custom scenario services.

11.3 Management Summary

The project should continue to emphasize:

- Real-time performance measurement.
- CARLA/ROS environment repeatability.
- Scenario configuration completeness.
- Vehicle test safety.
- Documentation and reproducible validation results.

11.4 Final Assessment

Sylvan is technically valuable and economically reasonable as a research prototype with real-vehicle demo evidence. Its greatest contribution is providing a practical path for non-intrusive virtual-real autonomous driving evaluation. The next stage should focus on repeatable latency measurement, standardized vehicle integration, automated smoke tests, reusable scenario packages, and controlled field-test cost management.